Laboratorinių darbų vertinimo kriterijai

Pirmasis laboratorinis darbas

Kriterijus	Paaiškinimas	Balų skaičius
Bent 3 taikomosios srities objektai ir operacijos su jais	Turi būti realizuoti API metodai bent **3** taikomosios srities objektams. Objektai turi būti prasmingi, sudaryti iš daugiau nei vieno lauko. Pvz., jei kuriama kategorija, neužtenka saugoti tik jos pavadinimą – galima taip pat saugoti kategorijos aprašą, sukūrimo laiką. Sistemos naudotojas (naudotojų lentelė) nėra taikomosios srities objektas. Turi būti realizuotas *CRUD* (*Create*, *Read*, *Update*, *Delete*) ir *LIST* (sąrašo gavimas) kiekvienam objektui. Turi būti realizuotas puslapiavimas. Turi būti realizuotas filtravimas. Turi būti realizuota *hypermedia* bent esminiams elementams. Turi būti bent 1 panaudojimo atvejis (profilis, panelė (*dashboard*) ir pan.), kur grąžinamas resursas sukonstruojamas iš kelių (bent 2) esybių.	3
Objektų hierarchinis sąryšis	Objektai tarpusavyje turi būti susiję hierarchiniu ryšiu: ● Tema → Įrašas → Komentaras ● Kategorija → Skelbimas → Užklausa ● Testas → Klausimas → Atsakymas Objektai privalo būti susiję “grandinėle” (ryšys: vienas-su-daug). Hierarchijoje neturi	1

	būti naudotojų esybės (lentelės). Be to, neturi būti vienas–su–vienu ryšių. Taigi, **negalimi** tokie variantai: ● Komentaras ← Užsakymas → Sąskaita ↓ Dokumentas ● Klientas → Užsakymas → Adresas ● Žemynas → Šalis → Sostinė ● Automobilis → VIN → Registracija Kodėl to reikia: tam, kad galėtume pasitreniruoti realizuoti apimties valdymą (angl. *scoping*) naudojant URL.	
Paruošta testavimo aplinka	Turi būti paruošta testavimo aplinka, kurioje galima greitai išbandyti visus API prieigos taškus (angl. *endpoints*). Testavimui rekomenduojame naudoti ***Postman***, tačiau galite naudoti ir kitas priemones, kurios leidžia patogiai testuoti užklausas. Turi būti parengtas užklausų rinkinys, kuris leistų pratestuoti **visus** prieigos taškus (*endpoints*).	1
Duomenų bazė užpildyta prasmingais duomenimis	Kiekviena duomenų bazės lentelė, kuri yra susijusi su hierarchinio ryšio objektais, turi turėti bent 5 įrašus. Įrašai turi būti prasmingi: ne atsitiktiniai raidžių kratiniai ar tiesiog žodis *string*.	1
Grąžinami teisingi kodai	Priklausomai nuo pateikiamos užklausos, kiekvienas prieigos taškas (*endpoint*) privalo grąžinti teisingą atsako kodą: 404, 201, 204 ir kiti. Jokiais atvejais neturėtų būti grąžinamas 500.	2

REST semantikos laikymasis ir apimtis	URI sukurti laikantis rest principų: POST turėtų būti: *POST http://www.example.com/products* POST neturėtų būti: *POST http://www.example.com/createProduct* *POST http://www.example.com/products/create* GET turėtų būti: *GET http://www.example.com/customers/3/orders/9* GET neturėtų būti: *GET http://www.example.com/getOrder?id=9&cid=5* Analogiškai ir su visais kitais metodais. Bent vienas metodas privalo būti hierarchinis ir apimti visus 3 taikomosios srities objektus. Pvz.: GET http://www.example.com/customers/3/orders/9/items	2

Gynimo metu studentas turi žinoti, gebėti diskutuoti:

- REST principus ir kaip jie realizuoti arba nerealizuoti programoje (pvz., *stateless*, *cachable*, *uniform interface*). Ką suteikia vienas ar kitas principas.
- Esminius *status codes*. Gynimo metu reikės demonstruoti išskirtinius atvejus (kai nėra užklausiamo resurso, kai užklausa neteisinga ir kt.). Taip pat, gynimo metu dėstytojai specialiai mėgins iššaukti neteisingą programos elgesį.
- Savo programos architektūrą. Žinoti, kurioje vietoje aprašomi keliai, kur vykdomos su užklausa susijusios operacijos. **Gebėti pridėti papildomą taikomosios srities objektą ir realizuoti jam CRUD operacijas.**
- Kodėl pasirinkta viena ar kita DB valdymo sistema (susiję su bakalauriniu). Kokios galimos alternatyvos.
- Jei naudotas LLM, papasakoti, kaip buvo kuriama jo pagalba, kaip LLM buvo valdomas (jei buvo). LLM gynimo metu nebus galima naudoti.

Antrasis laboratorinis darbas

Kriterijus	Paaiškinimas	Balų skaičius
Integruota autorizacija naudojant OAuth arba JWT technologinius sprendimus	Atitinkami sistemos prieigos taškai (*endpoints*) turi būti pasiekiami tik prisijungusiems naudotojams. Žetono (*token*) viduje turi būti saugoma naudotojo informacija, tokia kaip rolė ir ID.	3
Pasirinkta tinkama tokenų atnaujinimo strategija	Turi būti naudojama žetonų (*tokens*) atnaujinimo strategija, tokia kaip *refresh token*. *Access* tipo žetonai turi galioti trumpai, Refresh - ilgiau. Naudotojui neturi reikėti nuolatos prisijungti iš naujo.	2
Realizuoti *register*, *login* ir *logout* metodai	Atsijungimo metodas turi panaikinti naudotojo žetonus arba pažymėti kaip nebegaliojančius. Galima vietinė autentifikacija arba naudojant išorinį tiekėją (pvz. Google, Facebook ar kt.). Arba abu variantai :) Žetonai turi būti valdomi kūrėjo pusėje. Galima pamėginti realizuoti dviejų faktorių autorizaciją. Už tai “respectas” ir +1 balas (kuriam nors	2

	laboratoriniam).	
Projekte realizuota autorizacija pagal naudotojo rolę ir ID. Yra bent 3 rolės.	Naudotojai gali naudotis tik tam tikrais kreipimosi taškais (*endpoints*). Pavyzdžiui, tik administratorius gali sukurti skelbimų kategorijas. Įgyvendintos apsaugos, kur rolė negali apsaugoti nuo kenkėjiškų veiksmų. Pvz., paprastas naudotojas neturi galėti ištrinti arba redaguoti kito naudotojo komentaro.	3

Gynimo metu studentas turi žinoti, gebėti diskutuoti:

- Kokia yra žetono (*token*) struktūra ir kokia struktūra realizuota pas jį konkrečiai. Turi gebėti parodyti kurioje programos vietoje sukuriamas žetonas ir kokia informacija (*claims*) į jį įeina. Kodėl naudinga į žetoną įtraukti rolę. Kaip galima/negalima nulaužti žetoną.
- Ar programos sukuriamas žetonas yra pasirašomas. Jei yra - kurioje programos vietoje tai atliekama; kokiu būdu/algoritmu pasirašoma; kur yra slapti arba viešosios kriptografijos raktai.
- Kokiu būdu perduodamas žetonas (*header*'yje, *cookie*'je ar kt.). Jei perduodamas naudojant *cookie* - parodyti, kad uždėti *secure* ir *http-only* flagai. Žinoti, kokie abiejų būdų privalumai ir trūkumai.
- Kaip programa apsaugota nuo pagrindinių grėsmių (injekcijų, teisių eskalacijos ir kt.). Kurioje vietoje tai realizuota.
- Su autorizacija susijusius statuso kodus. Reikės pademonstruoti jų veikimą.

Trečiasis laboratorinis darbas

Kriterijus	Paaiškinimas	Balų skaičius
Sąsaja leidžia panaudoti visus realizuotus API metodus	Trims pasirinktiems dalykinės srities objektams turi būti galima atlikti *CRUD* veiksmus naudojant vartotojo sąsają. Turi būti realizuota registracija, prisijungimas ir žetonų atnaujinimas. Duomenų įvedimui turi būti panaudoti bent 4 skirtingi elementai (pvz. *Input*, *textarea*, vieno ar daugelio pasirinkimų mygtukai, pasirinkimas iš sąrašo (*dropdown* ar *list*), *date*/*time*/*color picker*, *spinner* ir t.t.) Duomenys turi būti grąžinami json formatu (ne html).	2
Panaudoti paveiksliukai, vektorinės grafikos ikonėlės, *header* ir *footer* elementai, modaliniai langai.	Naudotojo sąsajoje turi būti naudojami paveiksliukai – jų dydis turi kisti priklausomai nuo lango dydžio. Meniu punktuose turi būti naudojamos papildomos ikonėlės. Sukūrimo arba redagavimo arba ištrynimo formoms panaudoti modalinius langus. Svetainė turi antraštę *header* ir *footer* elementus	2

	ten, kur jų reikia.	
Panaudotos animacijos	Animacijos gali būti rodomos duomenų grąžinimo metu, užvedus pelę ant kai kurių elementų.	1
Naudotojo sąsaja yra vientisa ir pagyvinta naudojant CSS	Turi būti naudojama vientisa spalvų paletė ir stilistika. Elementai puslapyje turi būti išdėstyti logiška, naudotojui patogia seka. Turi būti panaudotas nestandartinis šriftas. Turi būti naudotojui teikiamas grįžtamasis ryšys apie jo veiksmus. Naršyklės *alert* langai tam neturi būti naudojami.	2
Responsive layout	Naudotojo sąsaja turi prisitaikyti prie skirtingų ekrano dydžių. Turi būti panaudotas bent vienas lūžio taškas (*breakpoint*). Turi būti realizuotas “Hamburgeris” mobiliai versijai.	1
Publikavimas	Projektas turi būti patalpintas (*hosted*) ir pasiekiamas internetu. Gali būti panaudotas bet kuris tiekėjas – *Azure*, *DigitalOcean*, *Render* ar kiti.	2

	Backend ir *Frontend* talpinimui skiriama po balą.	

Gynimo metu studentas turi žinoti, gebėti diskutuoti:

- Kodėl pasirinktas vienas ar kitas klientinės dalies karkasas. Kokios galimos alternatyvos.
- Kokią architektūrą naudoja karkasas. Kaip karkase saugoma būsena.
- Kaip įtraukiami nestandartiniai šriftai. Kokiu būdu ir kur tai padaryta studento programoje.
- Kaip CSS realizuota animacija. Kur yra konkrečios animacijos kodas.
- Kokiomis CSS priemonėmis realizuotas breakpoint. Gebėti jį pakeisti.
- Kodėl naudinga naudoti vektorinės grafikos elementus. Kur yra padėti vektorinės grafikos elementai (jų šaltinis).

Inžinerinis projektas (ataskaita)

Kriterijus	Paaiškinimas	Balų skaičius
Ataskaitos struktūra, formatavimas ir turinys	Ataskaita turi būti prieinama *Github* arba įkelta į *Moodle*. Turi būti laikomasi pateiktos struktūros, turi būti aprašyti visi skyriai. **Jokio DI šlamšto (angl. *AI slop*).**	3
Diegimo diagrama	Diegimo diagrama atitinka realų/numatomą sistemos diegimo atvejį. Diagrama yra taisyklinga, paruošta naudojant UML standartą. **Jokio *plantUML*.**	2
Naudotojo sąsaja ir jos projektas	Pateikiami naudotojo sąsajos ekranvaizdžiai ir bent 3 naudotojo sąsajos *wireframe,* padaryti su *figma* arba kitu įrankiu.	2
OpenAPI specifikacija	*OpenAPI* specifikacija yra papildyta naudojimo pavyzdžiais, galimais rezultatais. *OpenAPI* specifikacija pateikta skaitomu formatu dokumente arba Github. **Galite naudoti LLM**.	2
Išvados	Išvados yra išsamios, sunumeruotos. Yra	1

	pasakoma ne tik, kas buvo padaryta, bet ir *kas iš to*. **Jokio DI šlamšto.**	

Pastaba: Ataskaitos ginti nereikia, tačiau norint sužinoti ataskaitos trūkumus ir turėti galimybę ją pataisyti geresniam įvertinimui, ataskaitą reikia parodyti laboratorinio užsiėmimo metu dėstytojui.